



UNIVERSIDADE FEDERAL DO CEARÁ
Campus Quixadá
Curso: Redes de Computadores
Disciplina: Redes

Aula 10 – Princípios da Transferência Confiável de Dados

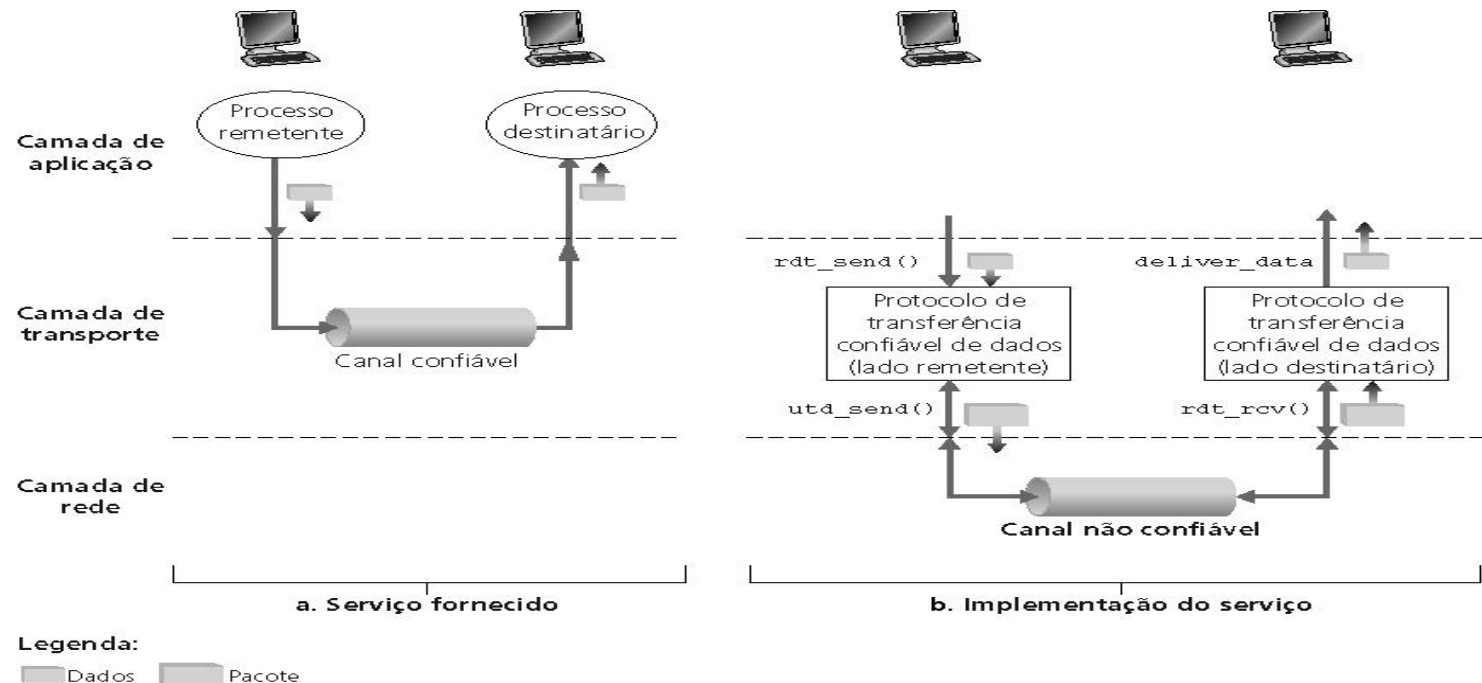
Prof. Rafael Braga

Conteúdo do Capítulo 3

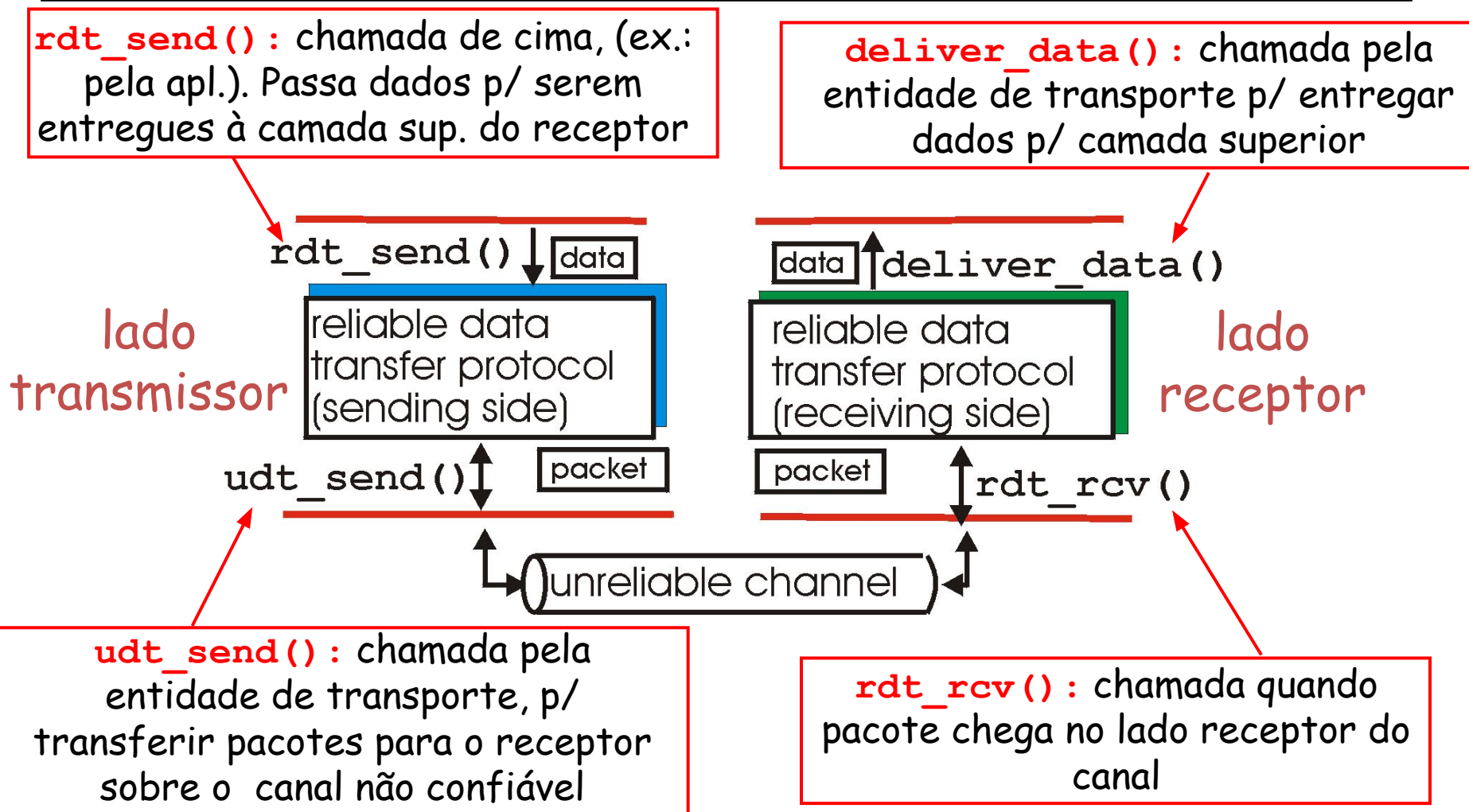
- 3.1 Serviços da camada de transporte
- 3.2 Multiplexação e demultiplexação
- 3.3 Transporte não orientado para conexão: UDP
- 3.4 Princípios da transferência confiável de dados
- 3.5 Transporte orientado para conexão: TCP
 - estrutura do segmento
 - transferência confiável
 - controle de fluxo
 - gerenciamento de conexões
- 3.6 Princípios de controle de congestionamento
- 3.7 Controle de congestionamento do TCP

Princípios de Transferência confiável de dados (rdt)

- importante nas camadas de transporte, enlace
- na lista dos 10 tópicos mais importantes em redes!
- características do canal não confiável determinam a complexidade de um protocolo de transferência confiável de dados (rdt)



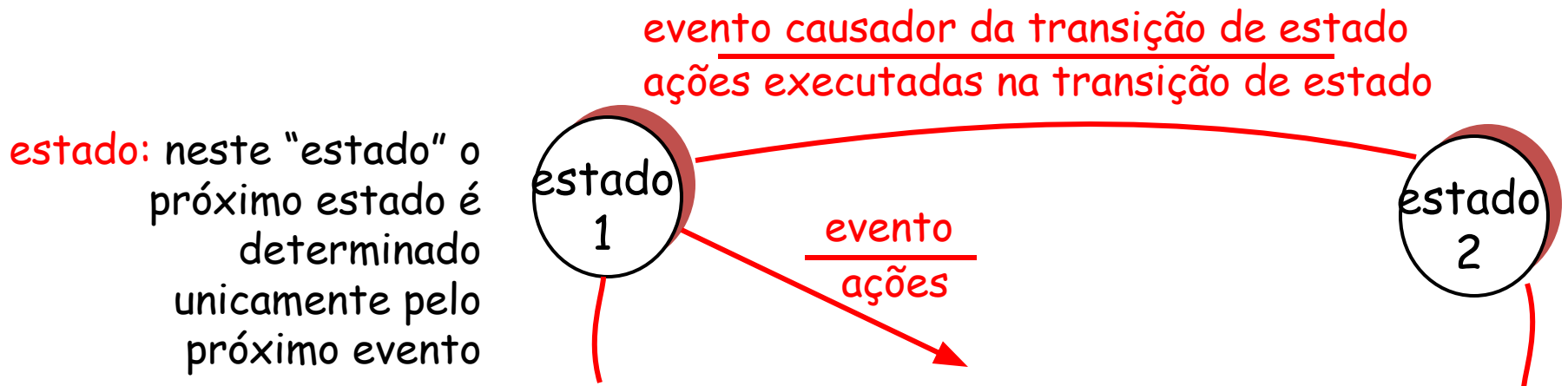
Transferência confiável: o ponto de partida



Transferência confiável: o ponto de partida

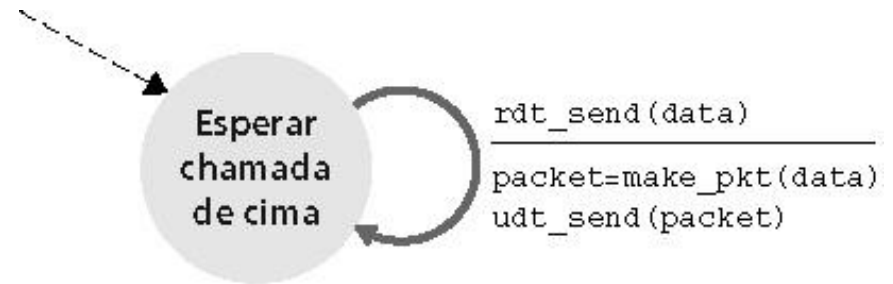
Iremos:

- desenvolver incrementalmente os lados transmissor e receptor de um protocolo confiável de transferência de dados (rdt)
- considerar apenas fluxo unidirecional de dados
 - mas info de controle flui em ambos os sentidos!
- Usar máquinas de estados finitos (FSM) p/ especificar os protocolos transmissor e receptor

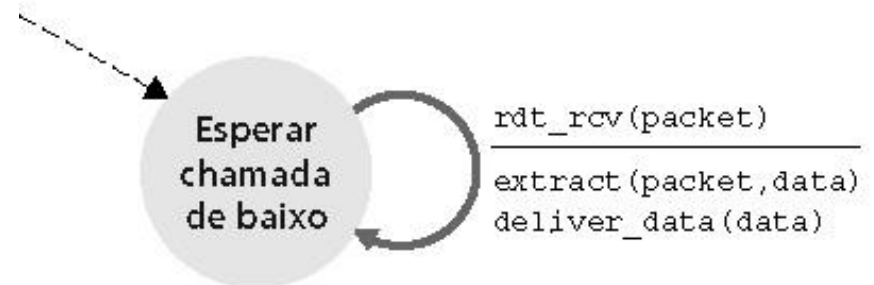


rdt1.0: transferência confiável sobre canais confiáveis

- canal de transmissão perfeitamente confiável
 - não há erros de bits
 - não há perda de pacotes
- FSMs separadas para transmissor e receptor:
 - transmissor envia dados pelo canal subjacente
 - receptor lê os dados do canal subjacente



a. rdt1.0: lado remetente

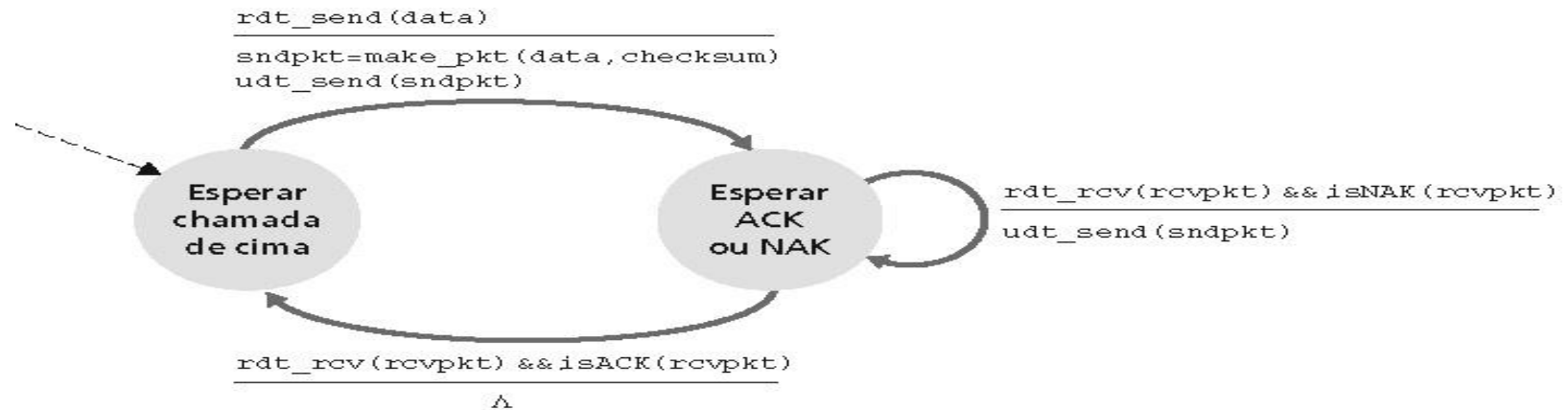


b. rdt1.0: lado destinatário

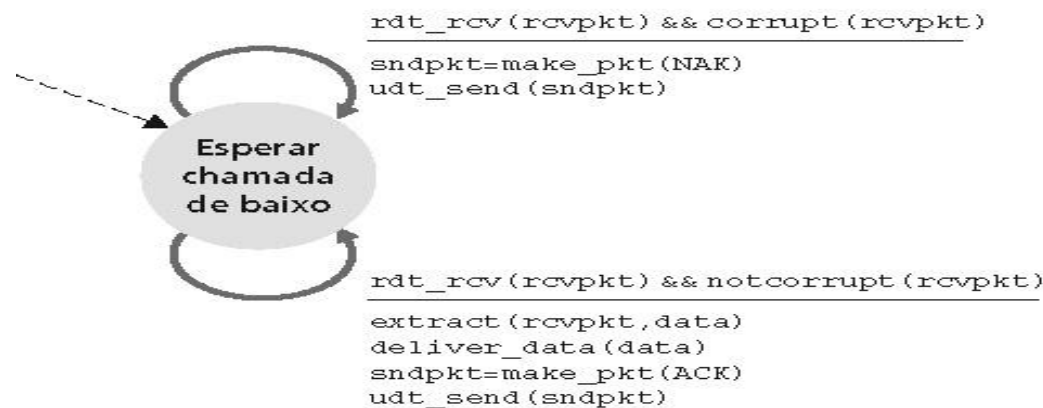
rdt2.0: canal com erros de bits

- canal subjacente pode trocar valores dos bits num pacote
 - lembre-se: checksum UDP pode detectar erros de bits
- a questão: como recuperar esses erros?
 - **reconhecimentos (ACKs)**: receptor avisa explicitamente ao transmissor que o pacote foi recebido corretamente
 - **reconhecimentos negativos (NAKs)**: receptor avisa explicitamente ao transmissor que o pacote tinha erros
 - transmissor reenvia o pacote ao receber um NAK
- novos mecanismos no **rdt2.0** (em relação ao **rdt1.0**):
 - detecção de erros
 - retorno ao transmissor: mensagens de controle (ACK,NAK)
receptor->transmissor

rdt2.0: especificação da FSM

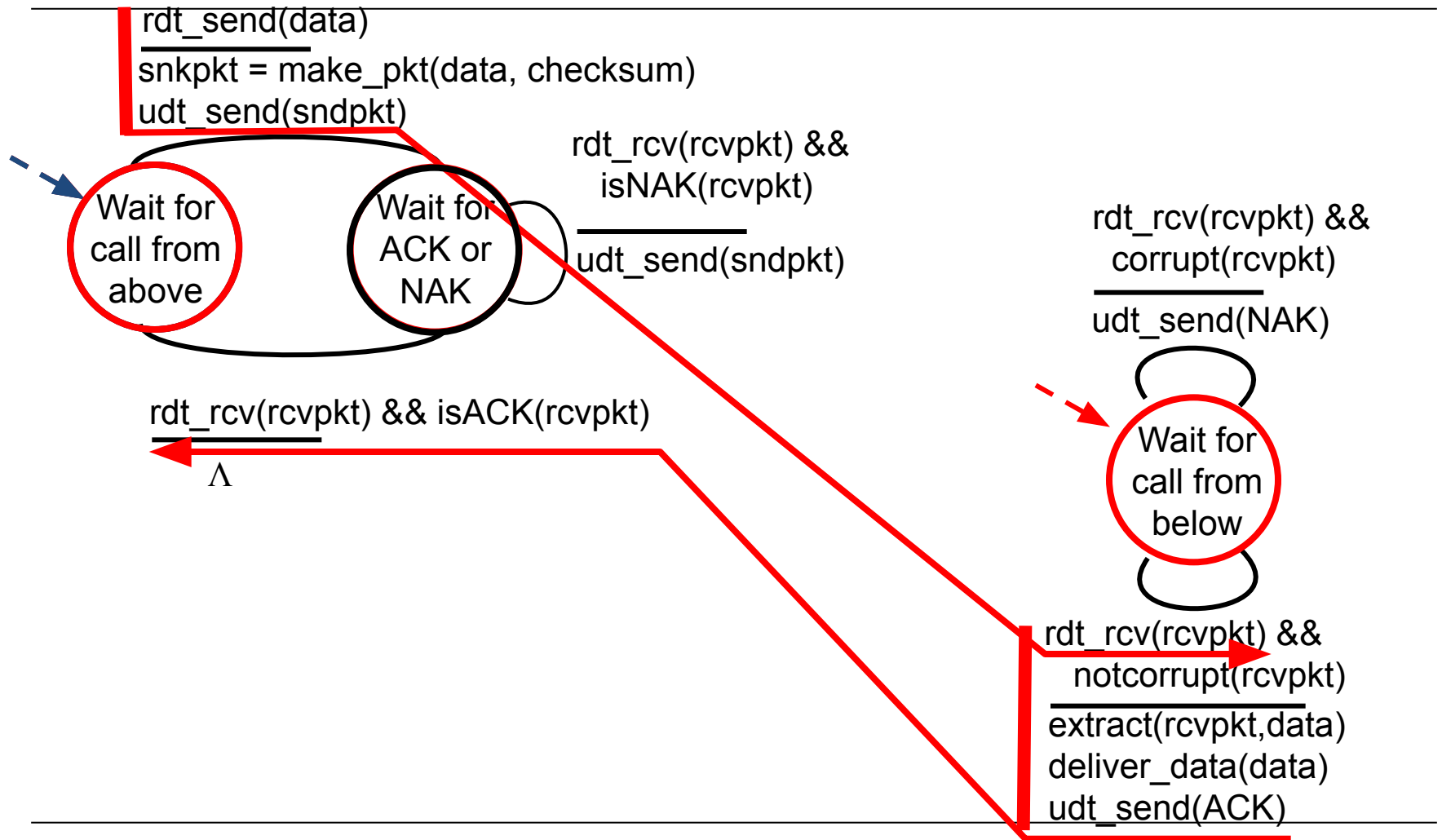


a. rdt2.0: lado remetente

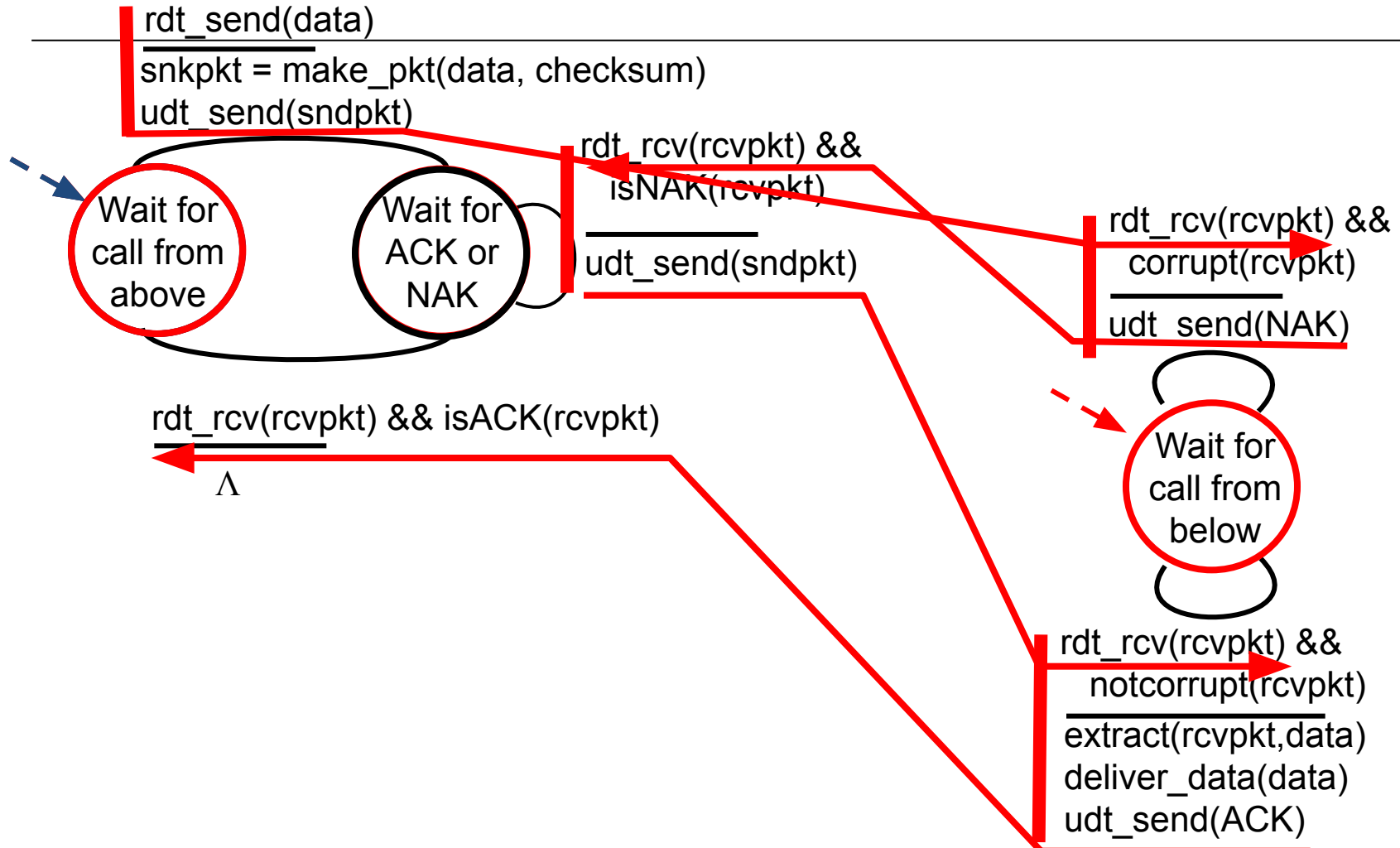


b. rdt2.0: lado destinatário

rdt2.0: operação com ausência de erros



rdt2.0: cenário de erro



rdt2.0 tem uma falha fatal!

O que acontece se o ACK/NAK for corrompido?

- Transmissor não sabe o que se passou no receptor!
- não pode apenas retransmitir: possibilidade de pacotes duplicados

O que fazer?

- remetente usa ACKs/NAKs p/ ACK/NAK do receptor? E se perder ACK/NAK do remetente?
- retransmitir, mas pode causar retransmissão de pacote recebido certo!

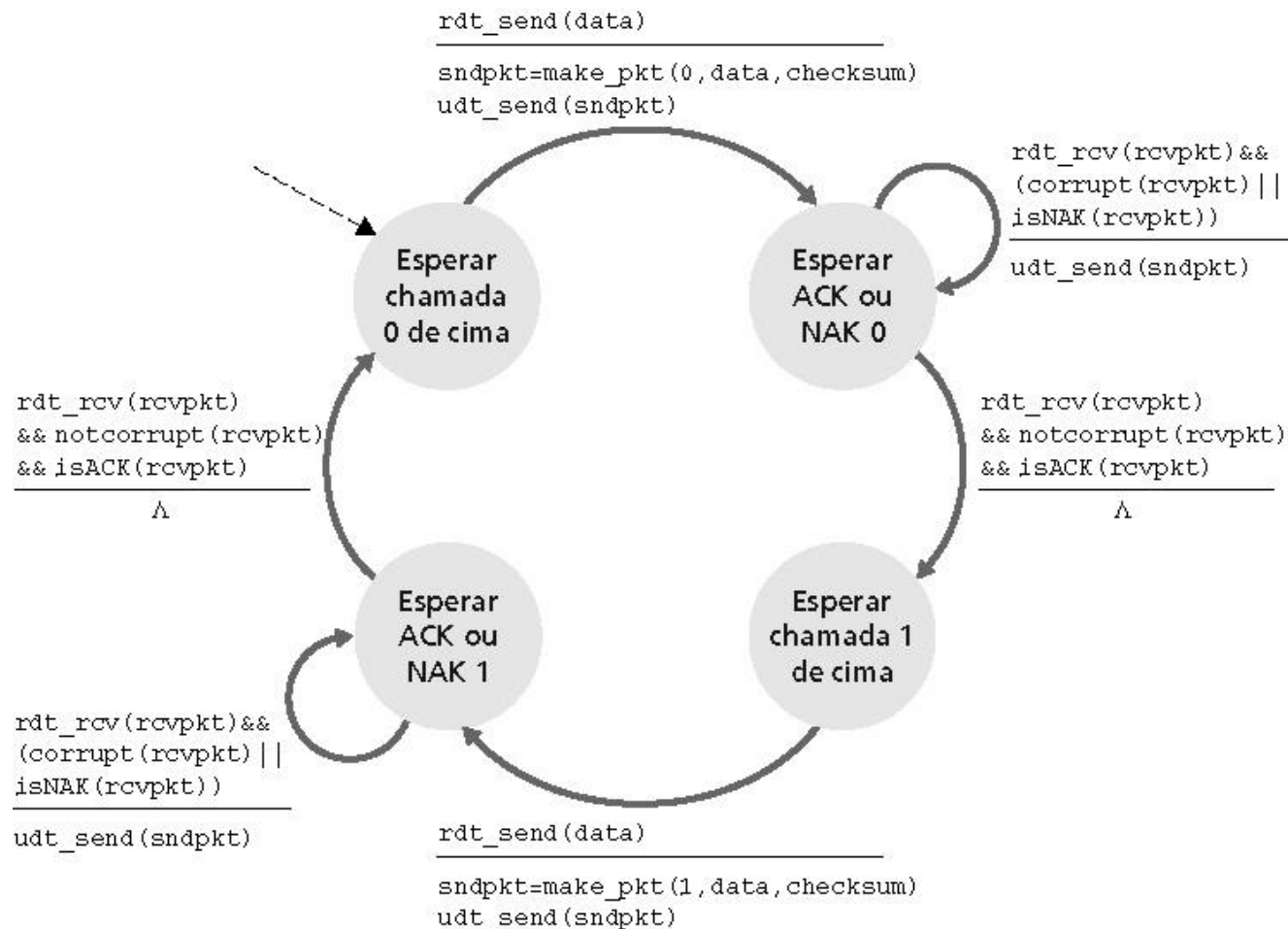
Lidando c/ duplicatas:

- transmissor inclui *número de seqüência* em cada pacote
- transmissor retransmite o último pacote se ACK/NAK chegar com erro
- receptor descarta (não entrega a aplicação) pacotes duplicados

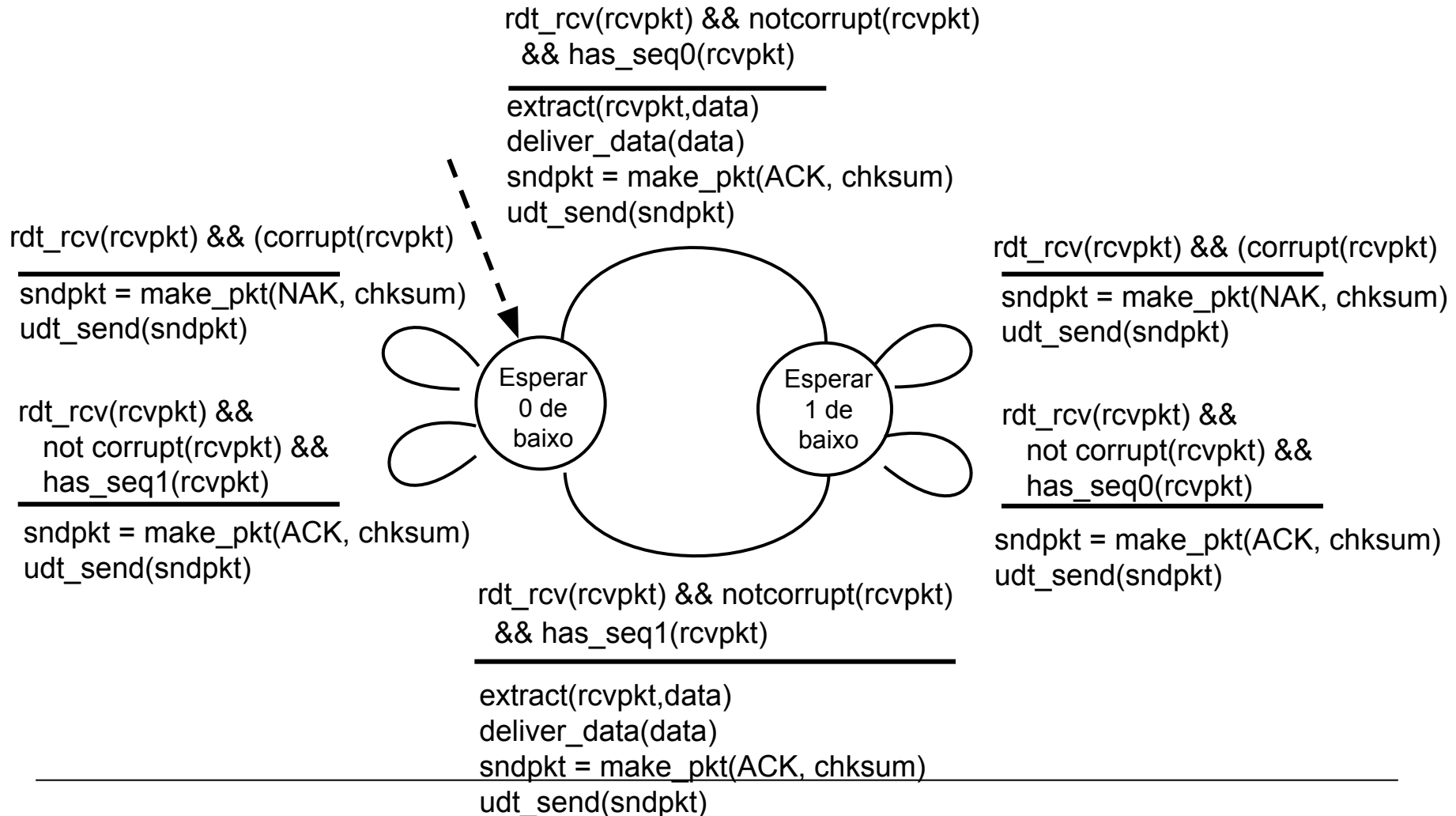
pare e espera

Transmissor envia um pacote, e então aguarda resposta do receptor

rdt2.1: transmissor, trata ACK/NAKs corrompidos



rdt2.1: receptor, trata ACK/NAKs corrompidos



rdt2.1: discussão

Transmissor:

- no. de seq no pacote
- bastam dois nos. de seq. (0,1). Por quê?
- deve verificar se ACK/NAK recebidos estão corrompidos
- duplicou o no. de estados
 - estado deve “lembrar” se pacote “corrente” tem no. de seq. 0 ou 1

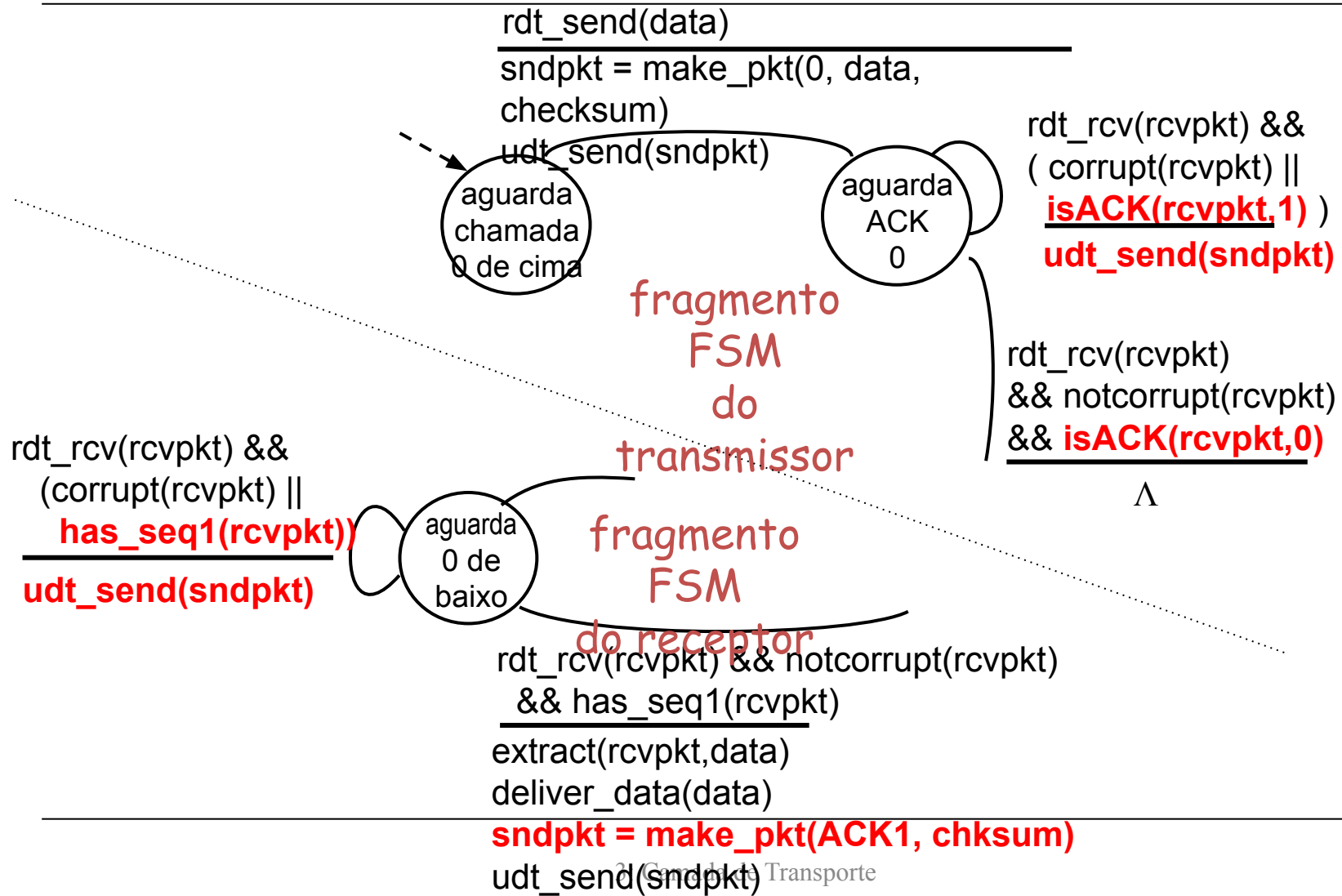
Receptor:

- deve verificar se o pacote recebido é duplicado
 - estado indica se no. de seq. esperado é 0 ou 1
- nota: receptor não tem como saber se último ACK/NAK foi recebido bem pelo transmissor

rdt2.2: um protocolo sem NAKs

- mesma funcionalidade do rdt2.1, usando apenas ACKs
- ao invés de NAK, receptor envia ACK para último pacote recebido sem erro
 - receptor deve incluir *explicitamente* no. de seq do pacote reconhecido
- ACKs duplicados no transmissor resultam na mesma ação do NAK: *retransmissão do pacote corrente*

rdt2.2: fragmentos do transmissor e receptor



rdt3.0: canais com erros e perdas

Nova hipótese: canal de transmissão também pode perder pacotes (dados ou ACKs)

- checksum, no. de seq., ACKs, retransmissões podem ajudar, mas não serão suficientes

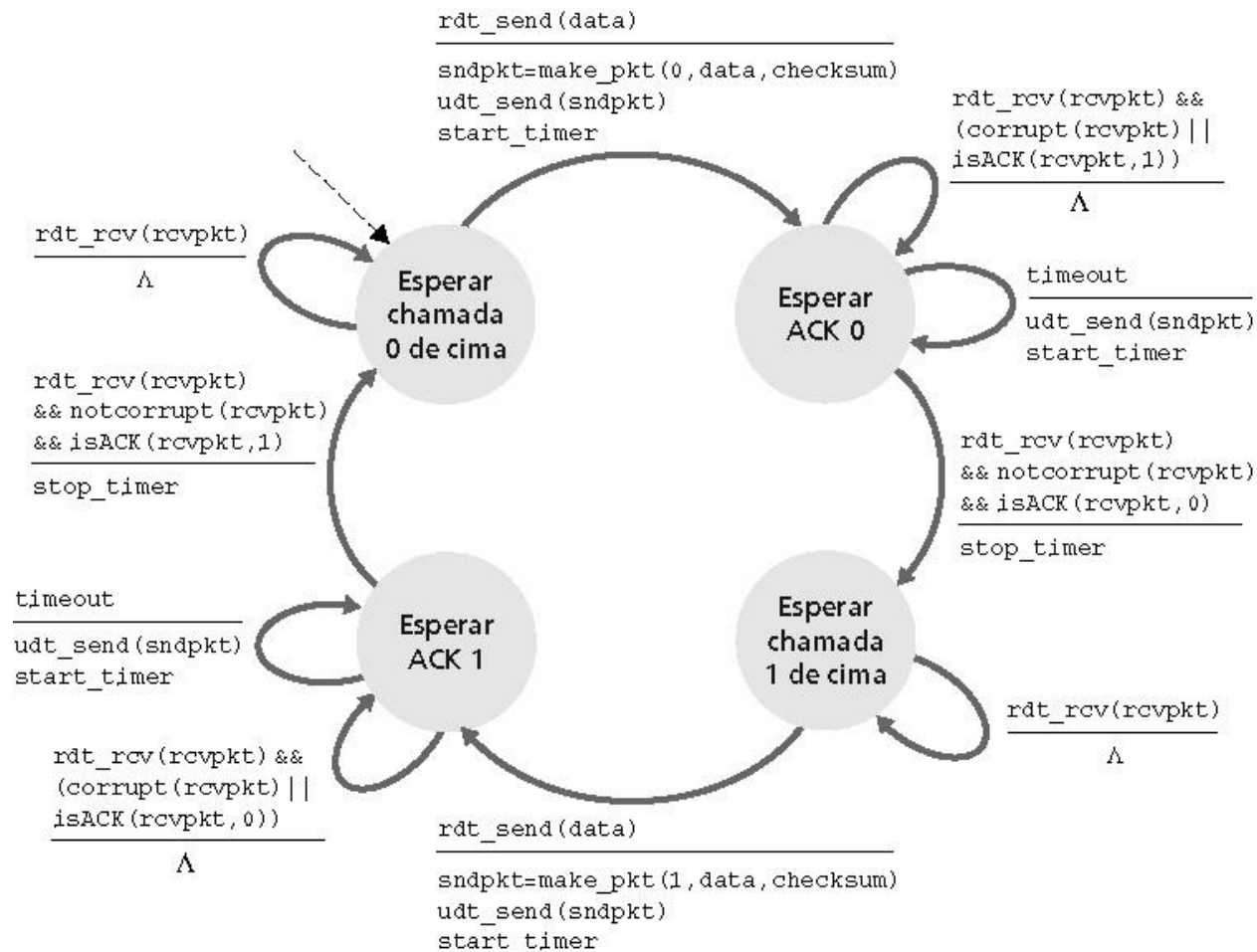
P: como lidar com perdas?

- transmissor espera até ter certeza que se perdeu pacote ou ACK, e então retransmite
- desvantagens?

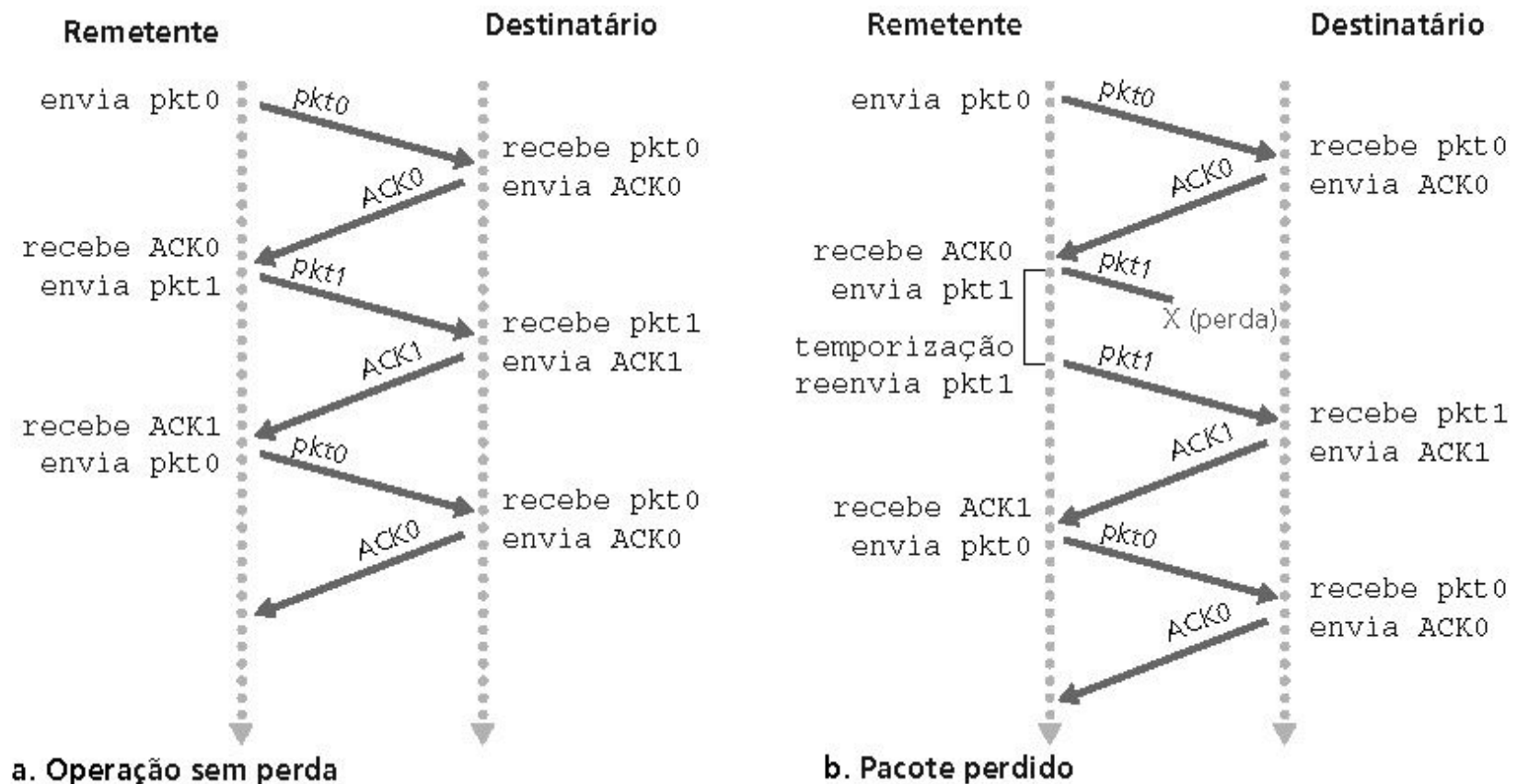
Abordagem: transmissor aguarda um tempo “razoável” pelo ACK

- retransmite se nenhum ACK for recebido neste intervalo
- se pacote (ou ACK) apenas atrasado (e não perdido):
 - retransmissão será duplicata, mas uso de no. de seq. já cuida disto
 - receptor deve especificar no. de seq do pacote sendo reconhecido
- requer temporizador

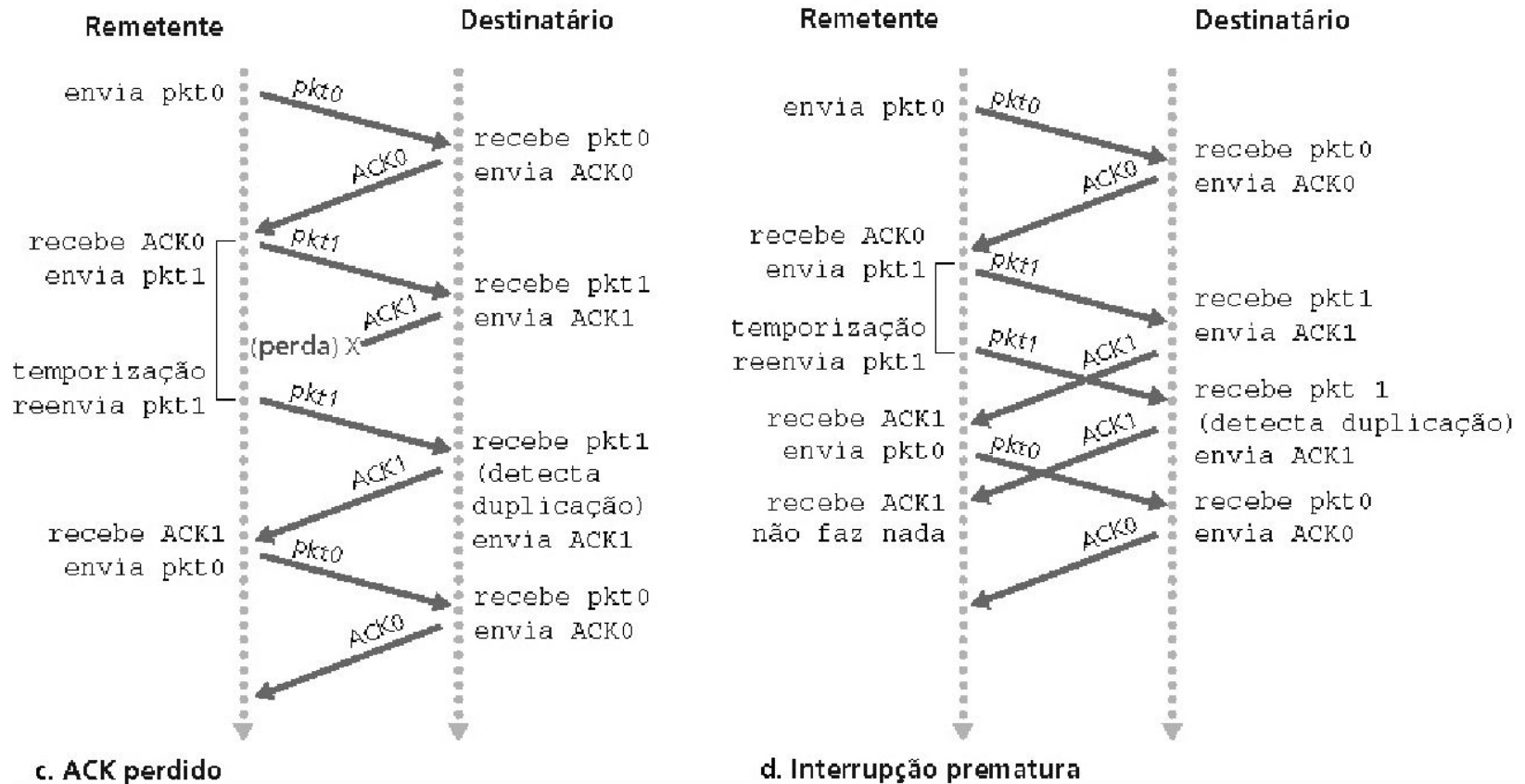
Transmissor rdt3.0



rdt3.0 em ação



rdt3.0 em ação



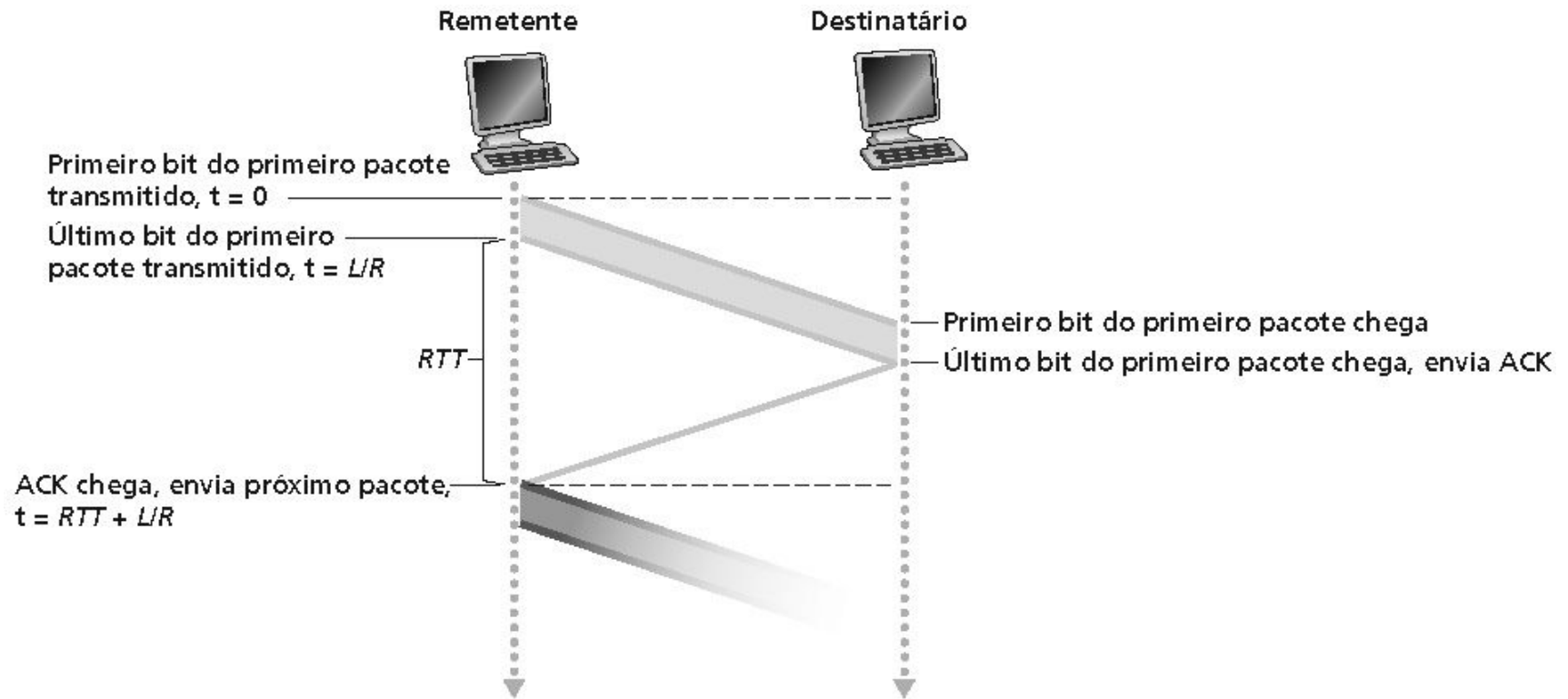
Desempenho do rdt3.0

- rdt3.0 funciona, porém seu desempenho é sofrível
- Exemplo: enlace de 1 Gbps, retardo fim a fim de 15 ms, pacote de 1KB:

$$d_{trans} = \frac{L}{R} = \frac{8000\text{bits}}{10^9\text{bps}} = 8\text{microsegundos}$$
$$U_{sender} = \frac{L/R}{RTT + L/R} = \frac{0.008}{30.008} = 0.00027$$

- r pac. de 1KB a cada 30 mseg -> vazão de 33kB/seg num enlace de 1 Gbps
- r protocolo limita uso dos recursos físicos!

rdt3.0: operação *pare e espere*



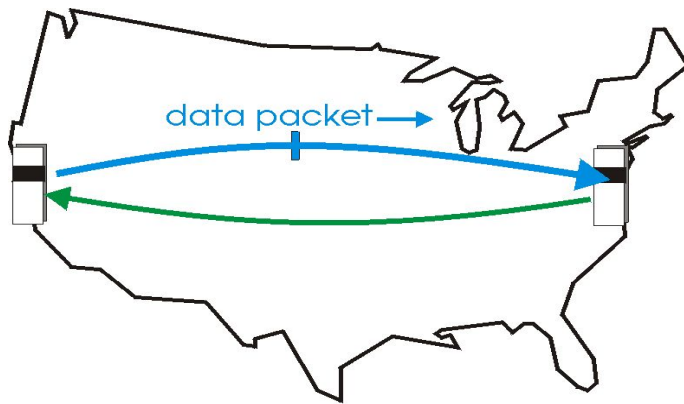
a. Operação *pare e espere*

$$U_{tx} = \frac{L/R}{RTT + L/R} = \frac{0,008}{30,008} = 0,00027$$

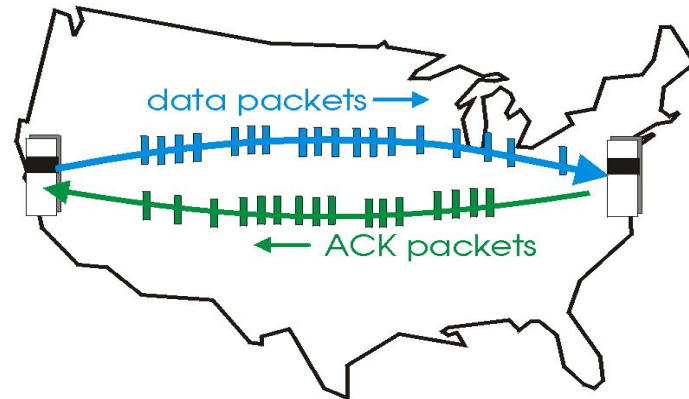
Protocolos com paralelismo (*pipelining*)

Paralelismo (*pipelining*): transmissor envia vários pacotes em seqüência, todos esperando para serem reconhecidos

- faixa de números de seqüência deve ser aumentada
- Armazenamento no transmissor e/ou no receptor



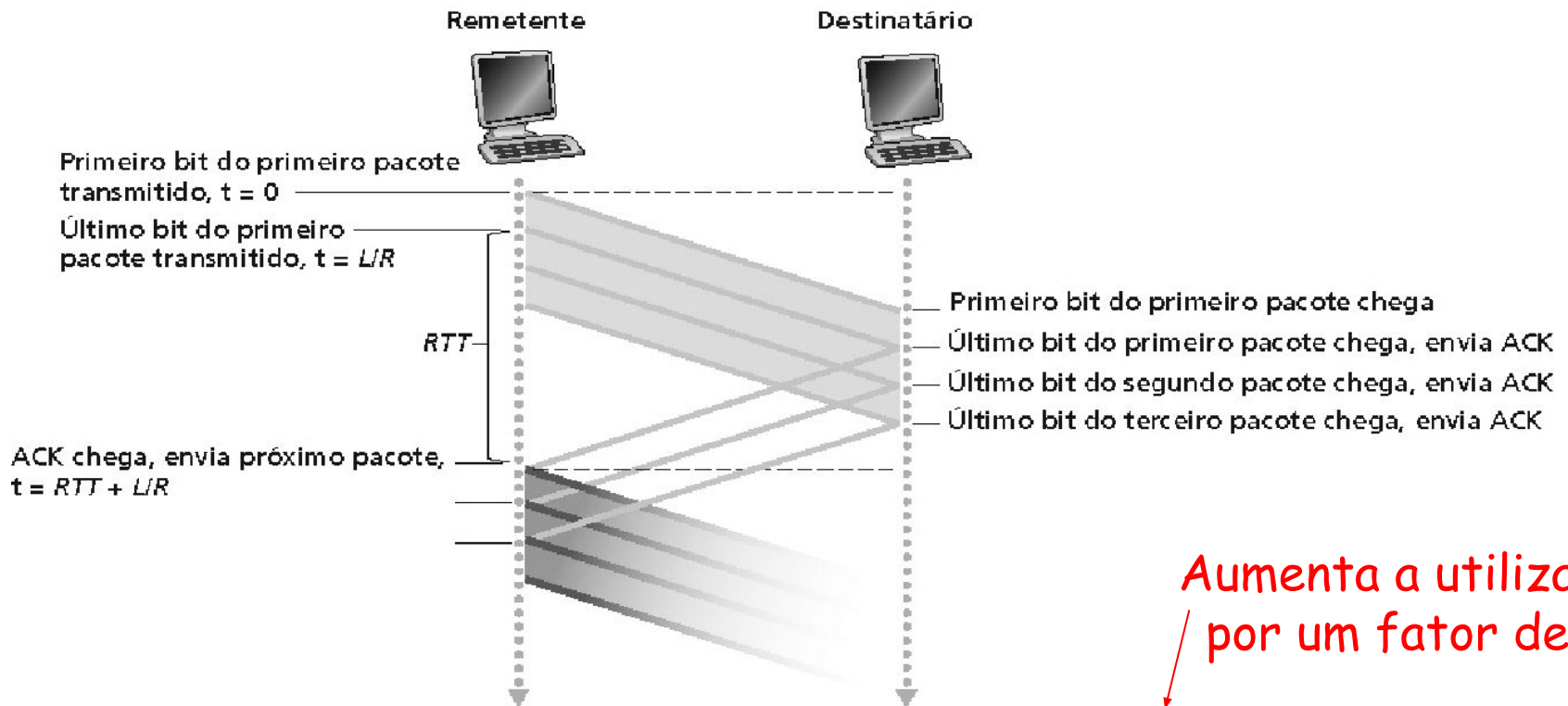
(a) operação do protocolo pare e espere



(a) operação do protocolo com paralelismo

- Duas formas genéricas de protocolos com paralelismo:
Go-back-N, retransmissão seletiva

Paralelismo: aumento da utilização



b. Operação com paralelismo

$$U_{tx} = \frac{3 \times L/R}{RTT + L/R} = \frac{0,024}{30,008} = 0,0008$$

Protocolos com Paralelismo

Go-back-N:

- O transmissor pode ter até N pacotes não reconhecidos no “tubo”
- Receptor envia apenas acks cumulativos
 - Não reconhece pacote se houver falha de seq.
- Transmissor possui um temporizador para o pacote mais antigo ainda não reconhecido
 - Se o temporizador estourar, retransmite todos os pacotes ainda não reconhecidos.

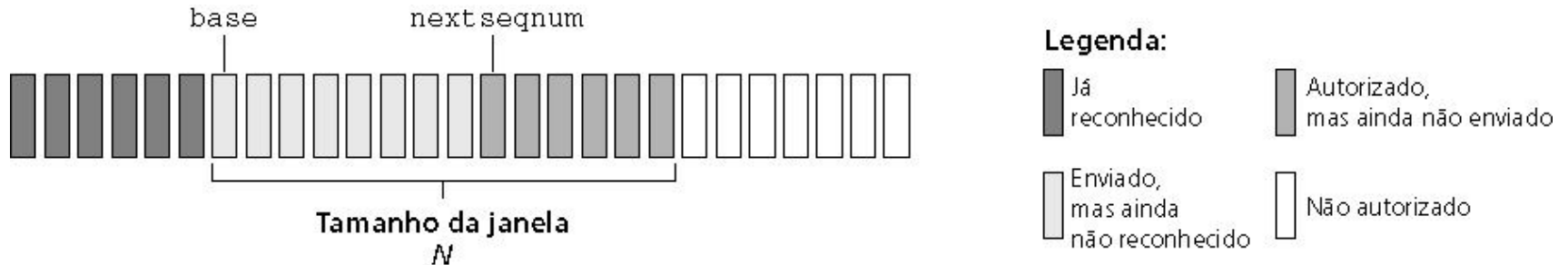
Retransmissão seletiva:

- O transmissor pode ter até N pacotes não reconhecidos no “tubo”
- Receptor reconhece pacotes individuais
- Transmissor possui um temporizador para cada pacote ainda não reconhecido
 - Se o temporizador estourar, retransmite apenas o pacote correspondente.

Go-back-N (GBN)

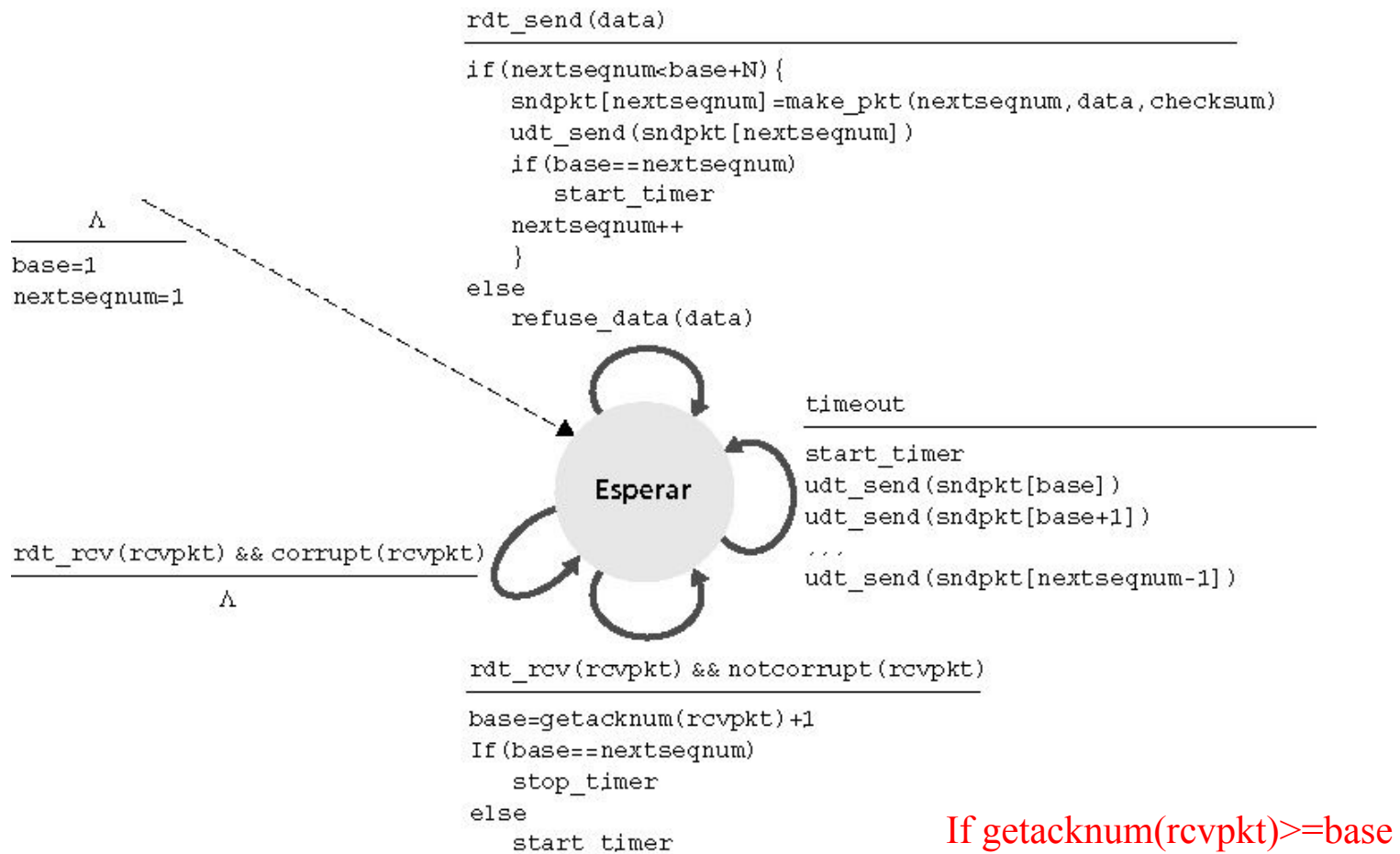
Transmissor:

- no. de seq. de k-bits no cabeçalho do pacote
- admite “janela” de até N pacotes consecutivos não reconhecidos



- $ACK(n)$: reconhece todos pacotes, até e inclusive no. de seq n - “ACK cumulativo”
 - pode receber ACKs duplicados (veja receptor)
- temporizador para pacote “mais antigo” enviado e não confirmado
- $timeout(i)$: retransmite pacote i e todos os pacotes com no. de seq maiores que estejam dentro da janela

GBN: FSM estendida para o transmissor



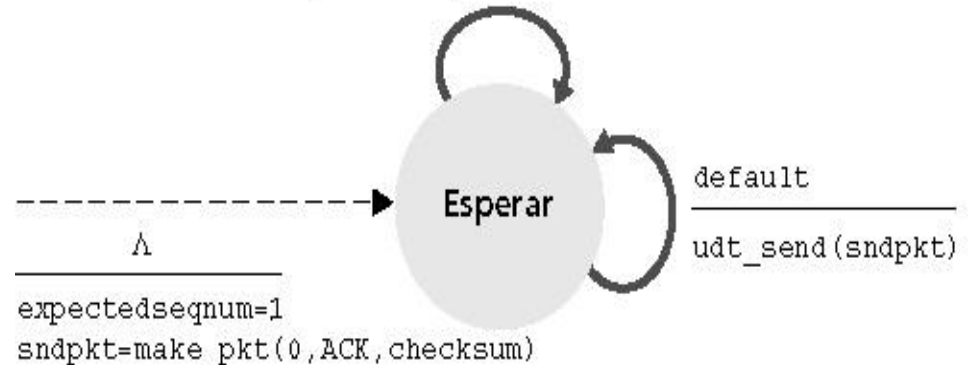
GBN: FSM estendida para o receptor

receptor simples:

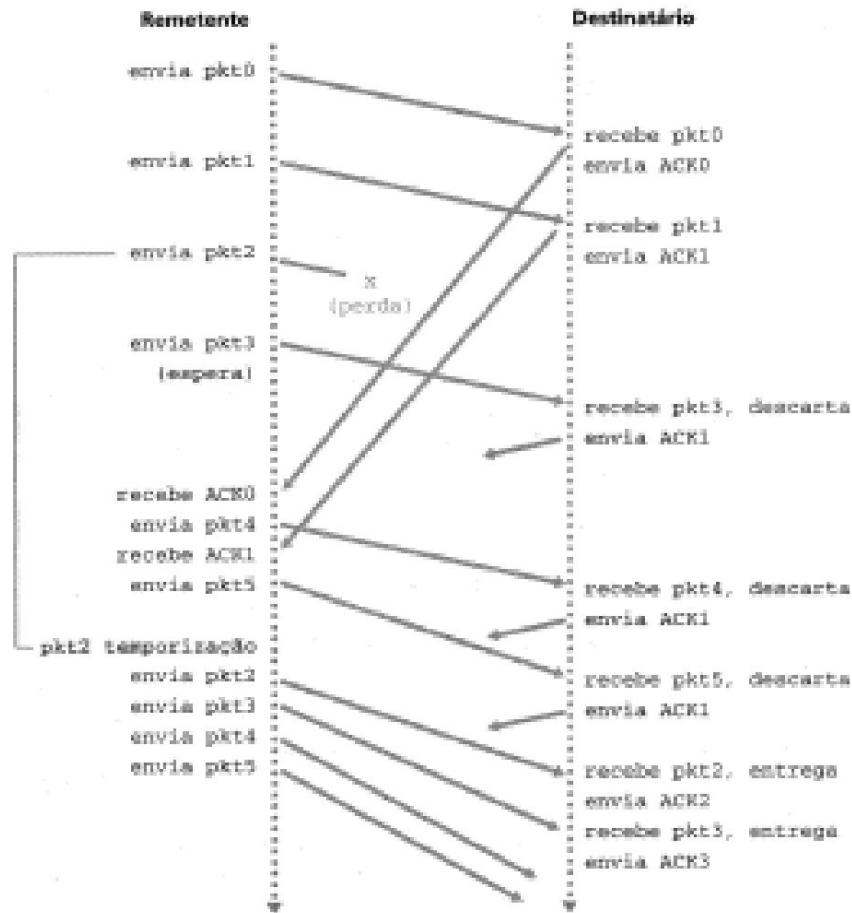
- usa apenas ACK: sempre envia ACK para pacote recebido corretamente com o maior no. de seq. *em-ordem*
 - pode gerar ACKs duplicados
 - só precisa se lembrar do **expectedseqnum**
- pacotes fora de ordem:
 - descarta (não armazena) -> **receptor não usa buffers!**
 - reconhece pacote com o mais alto número de seqüência em-ordem

```
rdt_rcv(rcvpkt)
    && notcorrupt(rcvpkt)
    && hasseqnum(rcvpkt, expectedseqnum)

extract(rcvpkt, data)
deliver_data(data)
sndpkt=make_pkt(expectedseqnum, ACK, checksum)
udt_send(sndpkt)
expectedseqnum++
```



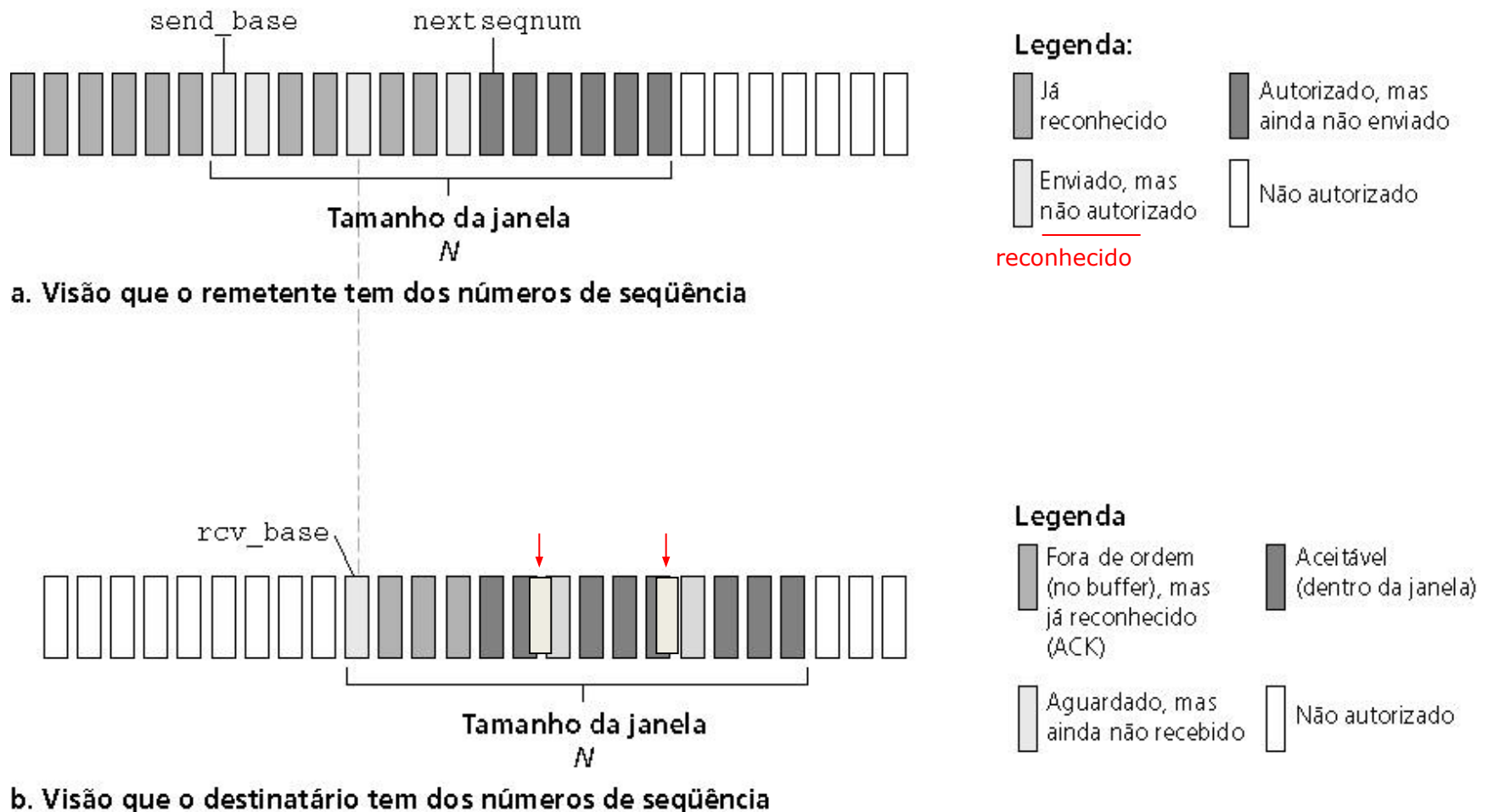
GBN em ação



Retransmissão seletiva

- receptor reconhece *individualmente* todos os pacotes recebidos corretamente
 - armazena pacotes no buffer, conforme necessário, para posterior entrega em-ordem à camada superior
- transmissor apenas re-envia pacotes para os quais um ACK não foi recebido
 - temporizador de remetente para cada pacote sem ACK
- janela do transmissão
 - N números de seqüência consecutivos
 - outra vez limita números de seqüência de pacotes enviados, mas ainda não reconhecidos

Retransmissão seletiva: janelas do transmissor e do receptor



Retransmissão seletiva

transmissor

dados de cima:

- se próx. no. de seq (n) disponível está na janela, envia o pacote e liga temporizador(n)

estouro do temporizador(n):

- reenvia pacote n, reinicia temporizador(n)

ACK(n) em [sendbase,nextseqnum-1]:

- marca pacote n “recebido”
- se n for menor pacote não reconhecido, avança base da janela ao próx. no. de seq não reconhecido

receptor

pacote n em

[rcvbase, rcvbase+N-1]

- r envia ACK(n)
- r fora de ordem: armazena
- r em ordem: entrega (tb. entrega pacotes armazenados em ordem), avança janela p/ próxima pacote ainda não recebido

pacote n em

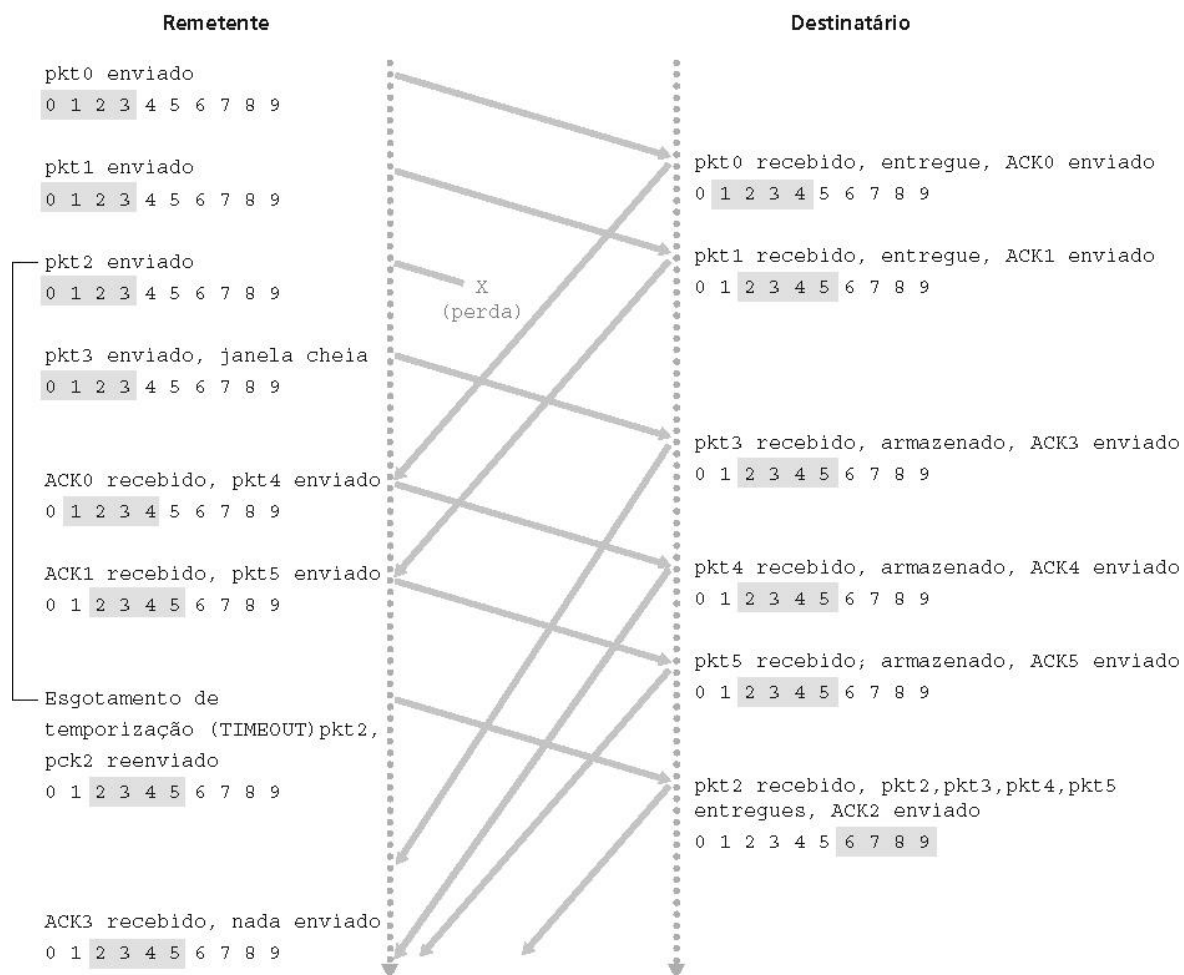
[rcvbase-N,rcvbase-1]

- r ACK(n)

senão:

- r ignora

Retransmissão seletiva em ação



Retransmissão seletiva: dilema

Exemplo:

nos. de seq : 0, 1, 2, 3

tam. de janela = 3

receptor não vê diferença entre os dois cenários!
incorretamente passa dados duplicados como novos em (a)

P: qual a relação entre tamanho de no. de seq e tamanho de janela?

